

基于文本挖掘和 SVM 的股票市场择时交易研究

中南财经政法大学

刘晔诚、田鹏飞、林海潮

摘要

近年来,随着计算机计算能力的提高以及人工智能的飞速发展,量化投资开始在中国金融市场崭露头角。借助于大数据的发展趋势,机器学习逐渐在量化投资领域发挥出优势。本文主要研究机器学习技术在量化投资领域的应用。

学术研究表明,投资者的情绪能够影响股票市场的未来走势。为合理量化投资者情绪,本文首先利用 Python 编写网络爬虫程序,在东方财富网“股吧”爬取了 200 万条标题文本数据,进行分词处理、词频统计后,得到初始关键词库。之后利用百度指数相关搜索词功能将关键词的个数扩充至 69 个,并通过 Elastic Net 和主成分分析法构造投资者情绪指数。同时,本文使用上证指数 2011 年 3 月 24 日至 2017 年 5 月 24 日共 1500 个交易日的数据,经过特征工程处理,建立了基于支持向量机(SVM)的股票市场择时交易模型,使用 Sliding Window 法进行最优参数的学习,从而对股市的未来趋势进行预测,得到未来每个交易日的操作信号。最后,将基于文本挖掘所构建的投资者情绪指数作为数据的一个特征代入 SVM 中,探讨其是否能提升模型的预测性能。

本文经过研究主要得到两个结论:一是经过特征工程对原始数据进行处理后,基于 SVM 所建立的股票市场择时交易模型对于上涨市、下跌市、震荡市均有较高的预测精度,且在震荡市捕捉到了更多的交易信息,相比买入并持有策略有更高的年化收益率。二是基于文本挖掘所构建的投资者情绪指数能在一定条件下提升模型对股票市场未来趋势的预测性能。

关键词: 文本挖掘; 投资者情绪指数; SVM; 择时交易

Abstract

In recent years, with the rapid development of computer computing power and artificial intelligence, quantitative investment began to emerge in China's financial markets. By means of the trend of Big Data, machine learning gradually play an advantage in the field of quantitative investment. The application of machine learning technology in quantitative investment field is mainly studied in this paper.

Academic research shows that sentiment of investor can affect the future trend of the stock market. In order to quantify the investor's mood reasonably, the Python software is used to write the web crawler program firstly, obtaining 2 million title text data in the Oriental wealth net “stock”. After the word processing and the statistic of the word frequency, the initial key word list is formed. Then related search word of the Baidu-index is used to expand the number of keywords to 69, and the investor sentiment index is computed through Elastic Net and principal component analysis. Meanwhile, using a total data of 1500 trading days of the Shanghai Composite Index from March 24, 2011 to May 24, 2017, the stock market timing trading model is built based on support vector machine (SVM), after the feature engineering processing. Then the optimal parameter is learnt by Sliding Window method, so as to predict the trend of the stock market and the operating signal of trading day in the future. Finally, the investor sentiment index based on text mining is substituted into SVM as a feature of the data, discussing if it can improve the predicting performance of the model.

Two conclusions are draw in this paper: First, after the processing of original data by the feature engineering, the stock market timing trading model based on SVM both has high prediction accuracy for the rising market, the falling market and the volatile market. Specially, it captures more of the trading information in volatile market. Compared to the “buy and hold” strategy, it has a higher annualized rate of return. Second, the investor sentiment index based on text mining can improve the predicting performance of the model.

Key words: Text Mining; Investor Sentiment Index; SVM; Timing Trade

一、 问题描述

（一） 研究背景

近年来, 互联网和人工智能技术飞速发展, 推动传统金融行业的大踏步前进, 尤其是量化投资领域。虽然目前中国市场的量化交易占总交易量的比重微乎其微, 但是量化投资因其内在交易机制的客观理性以及稳定收益的可实现性, 将成为金融投资领域未来的主流发展方向。

借助于计算机硬件的发展以及计算能力的显著提高, 加之创新性的新算法对计算速度的进一步提升, 机器学习的威力开始得以发挥, 国内一些私募基金开始将机器学习融入到自己的交易策略中。机器学习在量化投资领域有着独特的优势。对大多数量化交易而言, 主要是借助于计算机来构建大型的模型, 这些模型在某种意义上是静态的, 当市场发生变化, 效果或许会削弱, 需要人工调整去适应市场的变化。机器学习算法则能够高效分析大量的数据和特征, 并且在学习过程中自我改进以适应市场变化这部分因素。不会被情绪等主观因素干扰是机器学习的另一个优势。人的情绪会对预测的模式和判断产生某种影响, 不同情绪的影响叠加, 可能会产生截然不同的结果, 机器可以完美地克服这一问题。另一方面, 伴随着“大数据”的兴起, 尤其是金融领域的的数据优势, 为“数据为王”的机器学习技术提供了最佳应用场景。

本文顺应金融投资领域主流发展趋势, 结合大数据背景下的文本挖掘技术, 探讨机器学习在股票市场择时交易策略设计方面的可行性和有效性。

（二） 文献综述

本文主要借鉴了如下几篇国内外学者关于文本挖掘和机器学习用于量化投资研究的文献：

Kalev P S. et al(2004)应用朴素贝叶斯、KNN 和反向神经网络等机器学习方法, 在给定金融词典定义的情况下, 根据金融市场与关键词之间的关系生成概率规则, 对于规则技术关于股票的价格走势影响的有效性进行了研究, 研究结果证实了有效性并展示了其预测的准确性。Antweiler W.et al(2004)采用 SVM 算法分析了股票价格和华尔街日报流行专栏内容之间的关系, 研究发现媒体对于股价的悲观态度会显著导致股价下跌, 并且悲观程度会影响交易量的大小。Li F.(2010)通过对雅虎财经和 Raging Bull 网站上关于 45 家公司 150 万条消息的

挖掘，分析其对公司道琼斯指数的影响，发现股市评论可以对股票市场的价格波动进行预测，且交易量增加很大程度上和消息的分歧程度相关。张世军等(2013)通过对网络的评论进行爬取、分类获得网络舆情，结合股票的开盘、收盘价格构建起了基于网络舆情的 SVM 回归模型，并对股票价格的走势进行了预测。Junqu é de Fortuny E.et al(2014)根据 SVM 算法对文本进行分类，并利用 Python 对评论进行了标注，选用乖离率、心理线、威廉指标和强弱指标等技术指标验证了非理性指标预测股票价格走势的有效性。

二、 研究思路

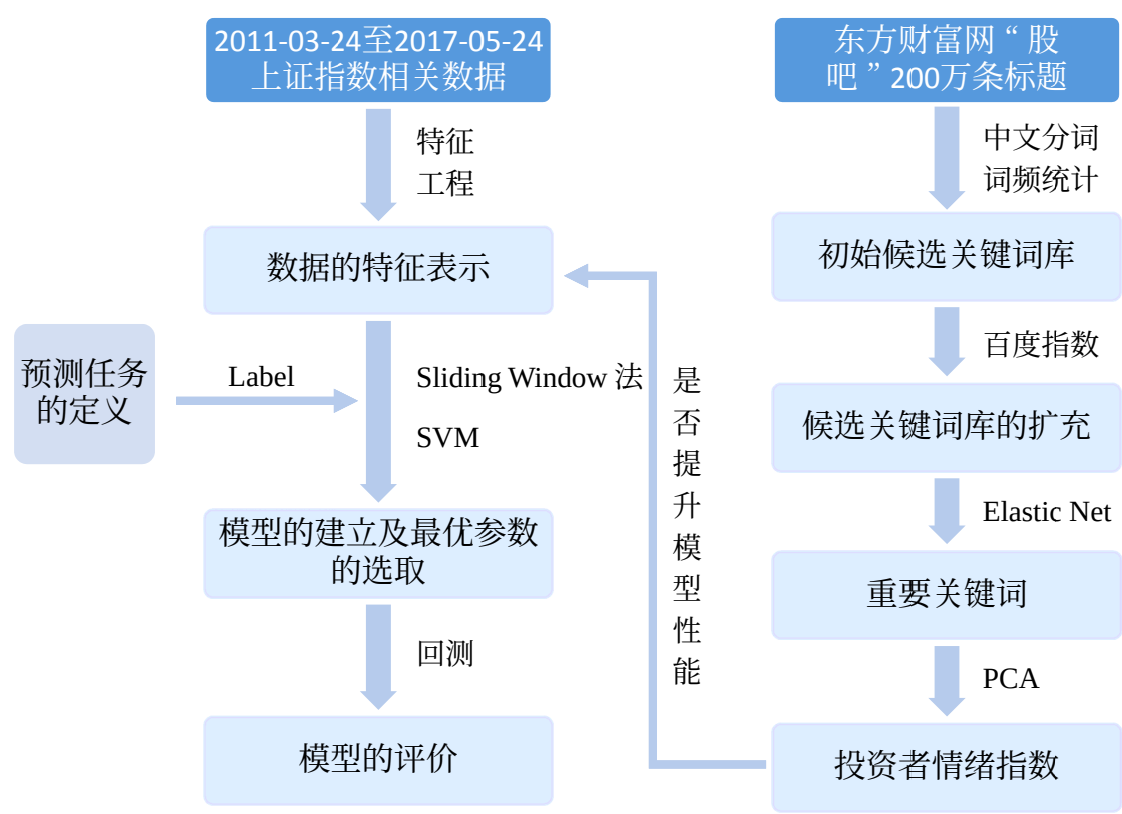


图 1：研究思路图

三、 数据来源与指标选择

(三) 数据来源

1. 用于投资者情绪指数构建所需数据来源

本文在投资者情绪指数的构建过程中所涉及的数据如表 1 所示：

表 1：数据来源表

数据内容	数据来源	数据量	数据用途
关于股票的挖掘文本	东方财富网“股吧”	200 万	关键词库的构建
关键词每日搜索量	百度指数	108261	情绪指数的构建

2. 用于股票市场择时模型构建所需基础数据来源

在基于 SVM 的股票市场择时交易模型的建立过程中, 我们使用的是 python 财经数据接口包 TuShare 提供的腾讯财经接口。TuShare 是一个免费、开源的 python 财经数据接口包, 主要实现对股票等金融数据从数据采集、清洗加工到数据存储的过程, 能够为金融分析人员提供快速、整洁和多样的便于分析的数据。由于上证指数反映了 A 股市场价格的总体变动趋势, 本文借助于该数据包得到了上证指数自 2011 年 1 月 1 日至 2017 年 6 月 9 日期间的开盘价、收盘价、最高价、最低价、成交量等基础数据。

(四) 软件说明

本文在数据处理和建模过程中用到的软件如表 2：

表 2：软件一览表

软件名称		软件用途
Python	爬虫程序	股票相关文本及百度指数的挖掘
	Scikit-learn	基于 Elastic Net 的重要关键词选取、 基于 SVM 的股票市场择时模型的建立及 相关训练特征的创建
	Jieba、wordcloud	文本分词、过滤、词频统计、词云
SPSS		基于 PCA 的投资者情绪指数的构建

(五) 百度指数

百度指数是以百度海量网民行为数据为基础的数据分享平台, 是当前互联网乃至整个数据时代最重要的统计分析平台之一, 自发布之日便成为众多企业营销决策的重要依据。借助于百度指数, 可以研究关键词搜索趋势、洞察网民需求变化、监测媒体舆情趋势、定位数字消费者特征; 还可以从行业的角度, 分析市场特点。

依照本文的研究目的, 我们用到了百度指数的两大功能: (1) 相关词分类中的搜索指数: 反映中心词所有相关词中搜索指数热门的关键词, 即与网民所搜索的某一关键词相关的其他关键词。我们选取与文本挖掘所得关键词相关的搜索热

度排名前十的词语，剔除掉与股票市场无关的词语，对初始关键词库进行扩充。

(2)指数趋势：包含 PC 搜索指数和移动搜索指数，由于 PC 搜索指数可从 2006 年 6 月 1 日查询，移动搜索指数只能从 2011 年 1 月 1 日开始查询，而实际使用的两者相加所对应的整体搜索指数，所以本文使用各关键词 2011 年 1 月 1 日至 2017 年 6 月 20 日的整体搜索指数。

四、 基于 Elastic Net 和 PCA 的投资者情绪指数的构建

(一) 基于词频的初始候选关键词库的构造

本文首先利用 Python 软件的 Requests 库、Re 库、bs4 库等编写“爬虫”程序，从东方财富“股吧”论坛爬取了 200 万条论坛标题的文本数据。选择东方财富“股吧”文本数据作为候选关键词的数据来源，一方面是考虑到东方财富网是中国乃至全球访问量最大、影响力最强的财经门户网站，在中国财经网站中，其多项统计数据位居第一；其次是因为东方财富“股吧”论坛的历史数据可追溯事件较长，数据量大，易于获取，对于关键词的提取意义重大。

得到文本数据之后，利用 Python 的 Jieba 中文分词库进行分词以及词频统计，对于无实际意义的词进行滤除，得到词频统计表(限于篇幅，仅展示词频排在前 20 的词语)如表 3 所示：

表 3：词频统计表（前 20）

词语	词频	词语	词频
公司	305732	散户	178921
市场	269402	股东	178021
股票	264657	涨停	177729
公告	242398	资金	159823
主力	239299	股市	143245
组合	219902	大盘	142098
股份	218061	反弹	132903
中国	211454	下跌	129876
股价	200235	实盘	127987
投资	199192	指数	119982

更清晰明了地展示词频统计的结果，利用 python 的 wordcloud 库可构建词频统计的词云图,如图 2 所示：



图 2：东方财富“股吧”标题文本词云图

(二) 利用百度指数进行候选关键词库的扩充

扩充后的词库可由表 4（共计 69 个词）进行描述

表 4：扩充后词库

公司	市场	股票	公告	主力	组合	股份	中国	股价	投资	散户	股东	涨停
资金	股市	大盘	反弹	下跌	实盘	指数	行情	走势	基金	买入	利好	黄金
石油	停牌	业绩	希望	减持	银行	科技	机会	新股	跌停	上市	操作	机构
证券	上涨	收购	筹码	重组	出货	分红	价值	板块	新三板	融资	回调	
原油	融资	风险	成本	解套	时间	技术	趋势	重大	清仓	持股	庄家	收盘
												收盘
												拉升
												尾盘
												龙头股
												模拟

(三) 爬取关键词库词语的百度指数

百度指数中整体指数对关键词的整体搜索量进行了比较好的描述，数据量大且较稳定，直接关系到关键词筛选以及情绪指数的构建。但从网页爬百度指数的每日搜索量数据并不简单，网站对于其原始数据进行了加密，无法直接取得。因此本文采用图像识别的间接爬取方式：利用 python 编写程序控制鼠标在浏览器上的移动，并即时下载其生成的图片并识别其中数字，最终获取每日数据。考虑到爬取速度的限制，本文采用了多线程编程爬取技术。

（四）基于 Elastic Net 的重要关键词的选取

为了保证我们所构建的投资者情绪指数与股票市场的价格波动高度相关, 需从候选关键词库中挑选出重要的关键词, 我们初步以上证指数的收盘价为响应变量, 对 69 个关键词对应的按日搜索量进行多元线性回归, 发现部分关键词之间存在多重共线性的问题。若使用常用的变量选择方法 LASSO, 将不会得到正确的结果。本文使用 Elastic Net 方法, 有效地克服了多重共线性的问题。Elastic Net 是一种将 LASSO 和 Ridge 回归的惩罚项线性组合在一起的正则化形式, 其目标函数为如下形式:

$$\min_{\omega} \left\{ \|y - X\omega\|_2^2 + \lambda_1 \|\omega\|_1 + \frac{\lambda_2}{2} \|\omega\|_2^2 \right\}$$

对于具有强相关变量组的数据, 其可以有效地将强相关变量组全部选入或全部剔除模型。

为便于最优参数的选取, 上述目标函数可改写为下式:

$$\min_{\omega} \left\{ \|y - X\omega\|_2^2 + \alpha \rho \|\omega\|_1 + \frac{\alpha(1-\rho)}{2} \|\omega\|_2^2 \right\}$$

其中 $\alpha = \lambda_1 + \lambda_2$, $\rho = \frac{\lambda_1}{\lambda_1 + \lambda_2}$ 。 α 越大, 对参数的惩罚力度越大, 对应越多

的参数趋于 0。 $\rho \in [0,1]$, 且 $\rho=1$ 时, Elastic Net 退化为 LASSO, $\rho=0$ 时, 退化为 Ridge 回归。

利用 Python 软件将上证指数收盘价作为响应变量, 将 69 个关键词的百度指数按日搜索量作为预测变量, 运用 Elastic Net 方法。使用 BIC 作为度量标准, 挑选出最优参数为: $\alpha = 2500$, $\rho = 0.2$ 。此时筛选出的重要关键词为 45 个, 剔除的 24 个关键词如表 5 所示。

表 5: 剔除的关键词

股东	利好	买入	股价	尾盘	机构	行情	收购	投资	回调	持股	反弹
涨停	分红	公司	重组	走势	筹码	出货	上涨	实盘	操作	解套	收盘

（五）基于 PCA 的投资者情绪指数的构建

主成分分析(PCA)是通过正交变换将一系列可能相关的变量转化为彼此线性

无关的变量(主成分)的统计学方法。同时,此正交变换保证第一主成分有着最大可能的方差,且下一个主成分在与前一主成分正交的条件下有着最大可能的方差。PCA 借助基于相关系数矩阵所对应特征值的方差贡献率,可挑选部分主成分以代替原始变量的绝大部分信息,从而实现降维的效果。

本文利用 SPSS 软件对基于 Elastic Net 筛选出的 45 个关键词所对应的按日搜索量进行主成分分析,在达到降维目的的同时,提取出所有关键词所反映的主要信息。按照特征值大于 1 的原则,选取了 7 个主成分分别为:

$$\begin{aligned} Z_1 &= 0.635x_1 + 0.219x_2 + \cdots + 0.392x_{69} \\ Z_2 &= 0.644x_1 + 0.132x_2 + \cdots + 0.108x_{69} \\ &\quad \dots \dots \\ Z_7 &= -0.005x_1 - 0.166x_2 + \cdots - 0.465x_{69} \end{aligned}$$

以各主成分所对应的方差贡献率为权重,计算 7 个主成分的加权和作为投资者情绪指数的代理值,计算结果如下所示:

$$invest_index = 41.37\%Z_1 + 10.965\%Z_2 + \cdots + 2.356\%Z_7$$

每个交易日对应的投资者情绪指数的具体取值见附件。

五、 基于 SVM 的股票市场择时交易模型的建立与检验

对于机器学习而言,股票市场交易是个具有巨大潜力的应用领域。由于大量历史数据的存在,人工对这些数据进行分析检测是很困难的,而机器学习技术对大数据具有先天的优势,可从中“学习”到隐含的模式或规律,进而指导人类作出有效决策。经济学有一个经典的有效市场假设,即股票价格反映了所有相关的信息,价格变化服从随机游走,对股价进行预测是没有意义的。但鉴于两个事实:(1)有效市场可能存在短暂的无效,(2)相关学术研究已表明,我国的股票市场尚未达到弱有效市场,故可利用股市的历史数据对未来趋势进行预测。该部分基于上证指数 2011 年 3 月 24 日至 2017 年 5 月 24 日共 1500 个交易日的相关数据,建立了可预测未来每天交易信号(买入、卖出、空仓或持有)的 SVM 模型,进而用于股票市场的择时交易。同时探讨了引入投资者情绪指数作为数据的一个新的特征后,模型的性能是否提升。

(一) 模型介绍—SVM 的核心思想及原理

支持向量机(Support Vector Machine)是 Cortes 和 Vapnik 于 1995 年首先提出的,其直观思想是:将原始数据映射到一个高维空间中,在新的特征空间里

寻找一个能够最大程度分隔两类样本的超平面。SVM 的建立以统计学习理论中的 VC 维理论和结构风险最小原理为基础，根据有限样本信息在模型的复杂度(对特定训练样本集的学习准确率)和学习能力(无错误地识别任意样本的能力)之间寻求最佳折中，以期获得最好的泛化能力。

支持向量机具有完备的理论基础和出色的学习能力，其借助于最优化方法在解决小样本、非线性及高维模式识别问题中表现出许多特有的优势。结构风险最小化原则决定了 SVM 的预测效果会优于神经网络等以经验风险最小化为优化目标的传统机器学习方法。

就其核心思想而言，支持向量机是基于 Mercer 核展开定理，通过非线性映射把原始特征空间映射到 Hilbert 空间，进而在新的特征空间上用线性学习方法解决非线性的分类和回归问题。本文使用较为常用的高斯核函数建立用于预测未来每个交易日具体交易类型的 SVM 模型。

给定训练样本集 $\{x_i, y_i\}_{i=1}^n$ ，其中 $x_i \in R^p$ ， $y_i \in \{1, -1\}$ ，SVM 所对应的优化问题为：

$$\begin{aligned} \min_{\omega, b, \zeta} & \frac{1}{2} \omega^T \omega + C \sum_{i=1}^n \zeta_i \\ \text{s.t. } & y_i (\omega^T \Phi(x_i) + b) \geq 1 - \zeta_i \\ & \zeta_i \geq 0, i = 1, 2, \dots, n \end{aligned}$$

对应的对偶问题为：

$$\begin{aligned} \min_{\alpha} & \frac{1}{2} \alpha^T Q \alpha - e^T \alpha \\ \text{s.t. } & y^T \alpha = 0 \\ & 0 \leq \alpha_i \leq C, i = 1, 2, \dots, n \end{aligned}$$

其中 $e = [1, 1, \dots, 1]^T$ ， $C > 0$ ， Q 为 $n \times n$ 的半正定矩阵，且有 $Q_{ij} = y_i y_j \cdot K(x_i, x_j)$ ，其中 $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ 为核，本文所使用的高斯核函数的表达式参见下式：

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) = \phi(x)^T \cdot \phi(x')$$

求解上述对偶问题可得 SVM 的决策函数：

$$f(x) = \text{sgn} \left(\sum_{i=1}^n y_i \alpha_i K(x_i, x) + b \right)$$

(二) 模型建立

1. 预测任务定义

由于通过模型精准预测未来每天的价格是一个极其困难的任务, 本文从另一个角度出发, 预测未来几天的市场变化趋势, 进而做出相应交易决策。若未来的趋势为上涨, 则我们在当天买入; 若趋势为下跌, 则我们在当天卖出; 若趋势不明显, 则继续空仓或持有。

假设我们的目的是预测在未来 K 天中价格的总体变化趋势, 且未来某天相对当天的价格变动超过 $p\%$ 时认为价格有较大波动, 下面我们给出具体的趋势量化方法:

假设每天的平均价格可以由下面公式来近似:

$$\bar{P}_i = \frac{C_i + H_i + L_i}{3}$$

其中, C_i 、 H_i 、 L_i 分别为第 i 天的收盘价、最高价和最低价。

设 V_i 代表由未来 K 天中每天的平均价格相对当天收盘价的百分比变化所构成的集合:

$$V_i = \left\{ \frac{\bar{P}_{i+j} - C_i}{C_i} \right\}_{j=1}^K$$

我们把百分比变化绝对值超过 $p\%$ 的价格波动进行累加作为一个指标变量 T :

$$T_i = \sum_v \{v \in V_i : |v| > p\%\}$$

指标 T 度量了在 K 天内日平均价格百分比变化明显高于目标变化的那些日期的变化之和, 大的正 T 值意味着未来 K 天中存在几天日平均价格相比当天收盘价的涨幅明显高于 $p\%$, 这种情况表明有良好的预期价格会上涨, 因此有潜在的机会发出买入指令。另一方面, 大的负 T 值表明价格在未来 K 有下跌的趋势, 因此在当天进行卖出操作。如果 T 的绝对值较小, 我们认为未来 K 天价格平稳波动或价格涨跌相互抵消, 因此继续空仓或持有, 不进行额外操作。

由于我们的真正目标是预测未来每个交易日的正确交易信号，下面通过引入一个特定的价格变化阈值 T^* ，将不同的 T 值转化为交易信号：

$$signal = \begin{cases} \text{卖出,} & T < -T^* \\ \text{空仓或持有,} & |T| < T^* \\ \text{买入,} & T > T^* \end{cases}$$

综上所述，最终的预测任务可定义为预测未来每个交易日的具体交易决策：买入、卖出还是空仓或持有，我们将其转化为一个具有三个类别的分类问题。

2. 特征工程

特征工程是将原始数据转化为特征，并根据所研究问题的领域知识提取、构建、删减或组合变化得到新的特征，以提升模型预测能力的过程。本文通过 Python 财经数据接口包 TuShare，获得了由上证指数 2011 年 1 月 1 日至 2017 年 6 月 9 日的收盘价、开盘价、最高价、最低价、成交量所构成的初始数据集。此数据集仅包含五个特征，对于影响因素众多、股价波动无明显特定规律的中国股票市场，明显无法得到一个好的预测效果。同时，在实际应用中机器学习算法的预测效果严重依赖于数据集特征表示的有效性。参考股票市场技术分析相关资料，我们选取表 6 所示的 35 个指标对初始数据集的特征进行扩展：

表 6：指标说明表

指标名称	指标说明
5 日移动平均价格 10 日移动平均价格 20 日移动平均价格	$MA(n)_t = \frac{\sum_{i=0}^{n-1} p_{t+i}}{n}$
5 日移动平均成交量 10 日移动平均成交量 20 日移动平均成交量	$MA(n)_t = \frac{\sum_{i=0}^{n-1} v_{t+i}}{n}$
价格相对前同一天的变化量	$\Delta p_t = p_t - p_{t-1}$
价格相对前一天的涨跌幅	$rate_t = \frac{p_t - p_{t-1}}{p_{t-1}}$
相对于过去 10 天中每一天的收益率	$before_rate_t = \frac{p_t - p_{t-i}}{p_{t-1}}, i = 1, 2, \dots, 10$
平均价格	$\bar{p} = \frac{p_H + p_L + p_C}{3}$

平滑异动移动平均线(MACD)	$EMA(n)_t = \frac{2}{n+1}p_t + \frac{n-1}{n+1}EMA(n)_{t-1}$ $DIF_t = EMA(12)_t - EMA(26)_t$ $DEA_t = \frac{2}{10}DIF_t + \frac{8}{10}DEA_{t-1}$ $MACD_t = DIF_t - DEA_t$
随机指标(KDJ)	$RSV_t = \frac{p_t - L_n}{H_n - L_n} \times 100$ $K_t = \frac{1}{3}RSV_t + \frac{2}{3}K_{t-1}$ $D_t = \frac{1}{3}K_t + \frac{2}{3}D_{t-1}$ $J_t = 3K_t - 2D_t$
顺势指标(CCI)	$\bar{p}_t = \frac{H_t + C_t + L_t}{3}$ $MA_t = \frac{\sum_{n=0}^{11} C_{t-n}}{12}$ $MD_t = \sum_{n=0}^{11} MA_t - C_{t-n} $ $CCI_t = \frac{1}{0.015} \times \frac{\bar{p}_t - MA_t}{MD_t}$
相对强弱指标(RSI)	$RSI(n) = \frac{A}{A+B} \times 100,$ 其中A为n日涨幅之和, B为n日跌幅之和的绝对值 这里n取14
变动速率(ROC)	$rate_t = \frac{p_t}{p_{t-n}}, \text{这里} n \text{取} 10$
OBV	$obv_t = \frac{2C_t - H_t - L_t}{H_t - L_t} \times V_t$

BULL	$middleband_t = \frac{\sum_{i=0}^9 p_{t-i}}{10}$ $md_t = \sqrt{\frac{\sum_{i=0}^9 (p_{t-i} - middleband_t)^2}{10}}$ $upperband_t = middleband_t + 2md_t$ $lowerband_t = middleband_t - 2md_t$
当日 K 线值 3 日 K 线调整值 6 日 K 线调整值	$Kline(n)_t = \frac{C_t - O_{t-n}}{H_{t-n} - L_{t-n}}$
6 日乖离线率(BIAS) 10 日乖离线率(BIAS)	$bias(n)_t = \frac{p_t - \bar{p}(n)_t}{\bar{p}(n)_t} \times 100\%, \bar{p}(n)_t \text{ 为 } n \text{ 日平均收盘价}$

依照第一节中预测任务定义的基准，我们取 K 为 10、 p 为 1、 T^* 为 15%，对每个数据样本进行类别标记。将未来 10 天有上涨趋势的数据样本标记为 1，表示当天进行买入操作；将有下跌趋势的数据样本标记为 -1，表示当天卖出；将无明显趋势的标记为 0，表示空仓或持有。

由于 SVM 对特征的尺度较为敏感，因此在进行正式的训练和预测之前要对每个特征进行标准化处理，使所有特征均具有 0 均值和单位标准差。

3. 训练集与测试集的划分

为检验所建立模型的有效性，将通过特征工程得到的数据集划分成训练集和测试集两部分。由于计算某些指标时需要用到当天数据的滞后期或超前期的数据，同时考虑到后续选取最优参数时验证集的有效合理划分，我们将 2011 年 3 月 24 日至 2015 年 3 月 10 日的数据归为初始训练集，用于 SVM 模型中初始最优参数的学习。进一步地，为有效评价所得模型的优劣，将 2015 年 3 月 11 日至 2017 年 5 月 24 日的数据归为测试集，以包含上涨、下跌和震荡市场类型。训练集和测试集的上证指数收盘价数据如图 3 和图 4 所示：

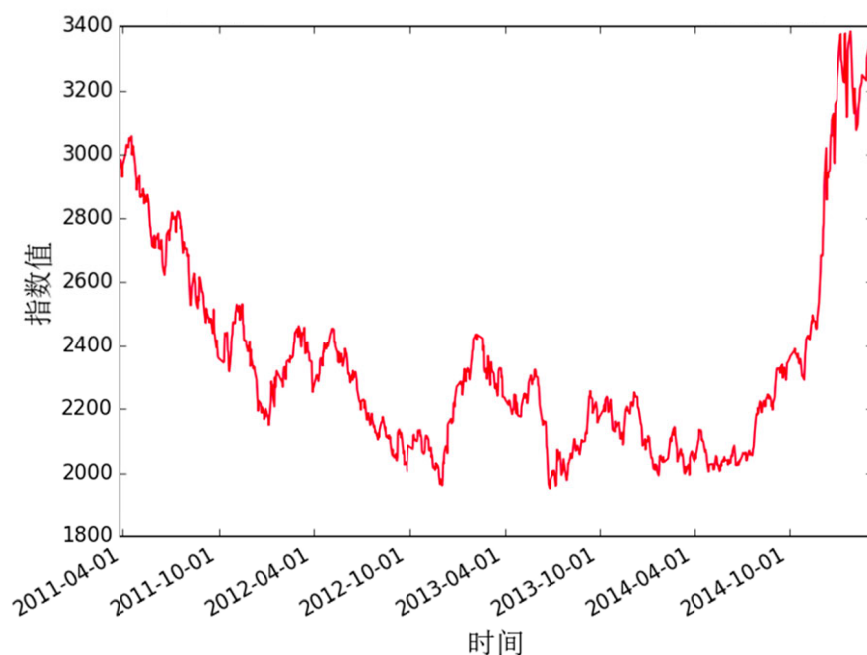


图 3：2011-03-24 至 2015-03-10 期间上证指数

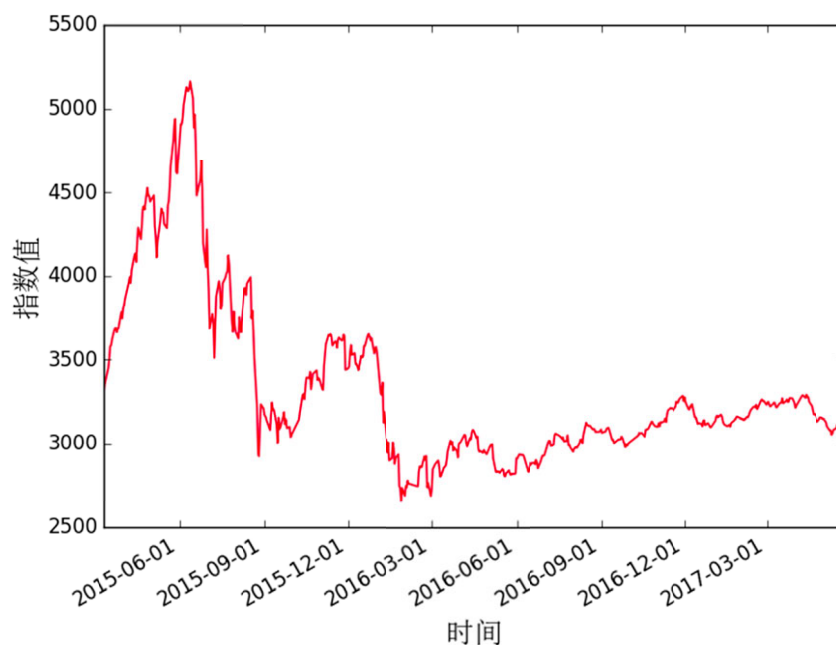


图 4：2015-03-11 至 2017-05-24 期间上证指数

4. 最优参数的选取—Sliding Window 法

由于我们所使用的上证指数数据集属于时间序列数据,对时间序列数据而言,不同时间的数据之间存在相关关系,且过去的的数据影响未来的数据取值,未来的数据不会影响过去的的数据取值。而传统的 K -折交叉验证方法对训练集进行 K 等分,共进行 K 次模型的训练和验证,其中在第 i 次的学习过程中,使用等分后的

第 i 份作为子训练集，其余 $K - 1$ 份作为子验证集。因此若将 K -折交叉验证方法用于基于时间序列数据的模型的参数择优，会出现使用未来数据预测过去交易类型(买入、卖出及空仓或持有)的情形，这违背了时间序列数据本身的逻辑。

对于如同本问题所研究的时间序列数据，普遍使用且有效的选取模型最优参数的方法是 Sliding Window 法。考虑到相对于当天的过去很长时间的数据对于未来数据的预测不起作用甚至可能产生干扰作用，当对未来某一固定跨度的时期进行预测时，仅选取过去固定数量的数据进行模型的训练。本文每次使用 240 个交易日(近似一年的时间段)的历史数据，预测未来 60 个交易日(近似一个季度的时间段)每天的具体交易类型，每进行训练预测一次后，将当前窗口向后移动 60 个交易日，进行新一轮训练和预测，直到遍及所有数据。原理如图 5 所示：

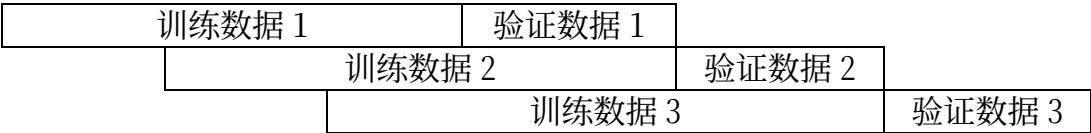


图 5：Sliding Windows 法示意图

本文使用 2011 年 3 月 24 日至 2015 年 3 月 10 日共 960 个交易日的数据样本作为初始训练集，选取用于未来 60 个交易日的预测模型的最优参数。当对未来新的 60 个交易日的模型进行最优参数选取时，添加已预测时间段的实际数据进行训练集的更新，重复以上过程直至对整个回溯期预测完毕。在每次最优参数的选取过程中，运用上述 Sliding Windows 方法可将训练集划分为多个子训练集。对于任意一组特定参数，每个子训练集中前 240 个数据样本用于该参数下模型的训练，后 60 个数据样本用于对应所得模型有效性的验证。最后将多个子验证集上的结果取平均，作为模型在该特定参数条件下的总体有效性指标。将平均性能最好的参数作为此次预测的最优参数。

（三）模型评价—回测

将每次学习得到最优模型用于未来 60 个交易日每天交易信号的预测，最终得到从 2015 年 3 月 11 日至 2017 年 5 月 24 日的所有预测结果。同时，为探讨本文所构建的投资者情绪指数对模型的预测性能是否有提升作用，我们将投资者情绪指数作为一个新的特征加入到 SVM 模型中，得到了一个新的预测结果，并对两结果进行比较分析。

1. 评价指标的选取

传统上评价一个机器学习算法性能可用准确率、错误率、精度、召回率、AUC 等衡量标准，但由于本文所建立的模型整体性能的衡量需与经济效益和风险相结

合且需重点考虑,所以这些评价标准不能直接应用到本文所研究的问题中。我们将使用表 7 所列指标对基于模型的交易策略进行评价。

表 7:评价指标说明表

评价指标	指标说明
累计收益率	$cum_profit(T) = \frac{total_money_T - 1}{1} \times 100\%$ 其中T为回测期总天数
年化收益率	$year_profit(T) = (1 + cum_profit)^{\frac{250}{T}} - 1,$ 其中T为回测期总天数, cum_profit为累计收益率
最大回撤	$max_retrace(T) = \min_t \left\{ \frac{total_money_t}{\max_i \{total_money_i\}_{i=1}^{t-1}} - 1 \right\}^T_{t=2}$
最大连续下跌次数	连续下跌的交易次数的最大值

2. 未加入投资者情绪指数的回测结果分析及评价

为检验本文所建立的基于 SVM 的股票市场择时交易模型对于实际交易的有效性,以 2015 年 3 月 11 日至 2017 年 5 月 24 日为回测期,选取初始资金为 1 万元,假设每次交易的费率为 0.03%,将模型预测结果所产生的累计总资产变动情况与在回测期第一天买入并一直持有到回测期末的情况作比较。本部分首先展示了模型在整个回测期的预测结果,之后将回测期进行划分,分别给出具有上涨、下跌和震荡趋势的时间段的测试情况。

(1) 整个回测期

该模型在整个回测期的预测结果如图 6 和表 8 所示：

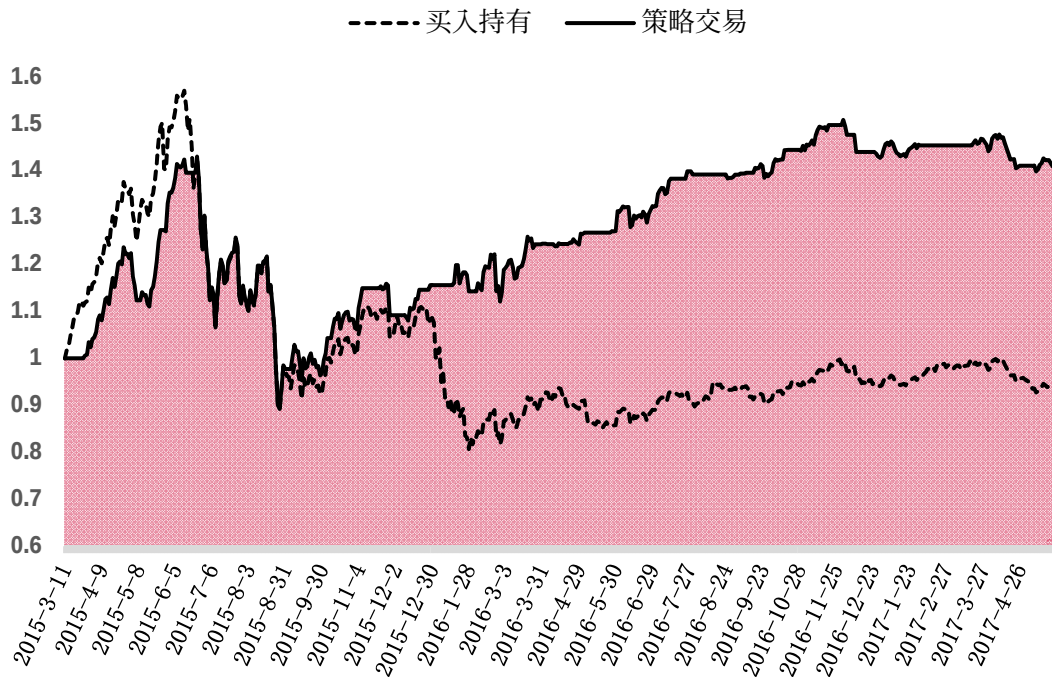


图 6：累计总资产变动情况图（2015-03-11 至 2017-05-24）

表 8：未加入指数整个区间回测结果

基本信息	预测天数：540	滑窗长度：60
交易类型	买入并持有	策略交易
初始资金	1 万元	1 万元
累计收益率	-6.89%	40.99%
最终余额	0.9311 万元	1.41 万元
年化收益率	-3.25%	17.24%
最大回撤	-48.60%	-37.59%
最大连续亏损次数	5	6

由图 6 和表 8 可以看出，对于整个回测期而言，相比于买入并持有操作所导致的 3.25% 的资金亏损，本文基于 SVM 所建立的股票市场择时模型取得了较高的 17.24% 的年化收益率，但最大回撤过高，对应风险较大。

（2）具有明显上涨趋势的回测期

在 2015 年 3 月 11 日至 2015 年 6 月 4 日的“牛市”期间，该模型的预测效果如图 7 和表 9 所示：

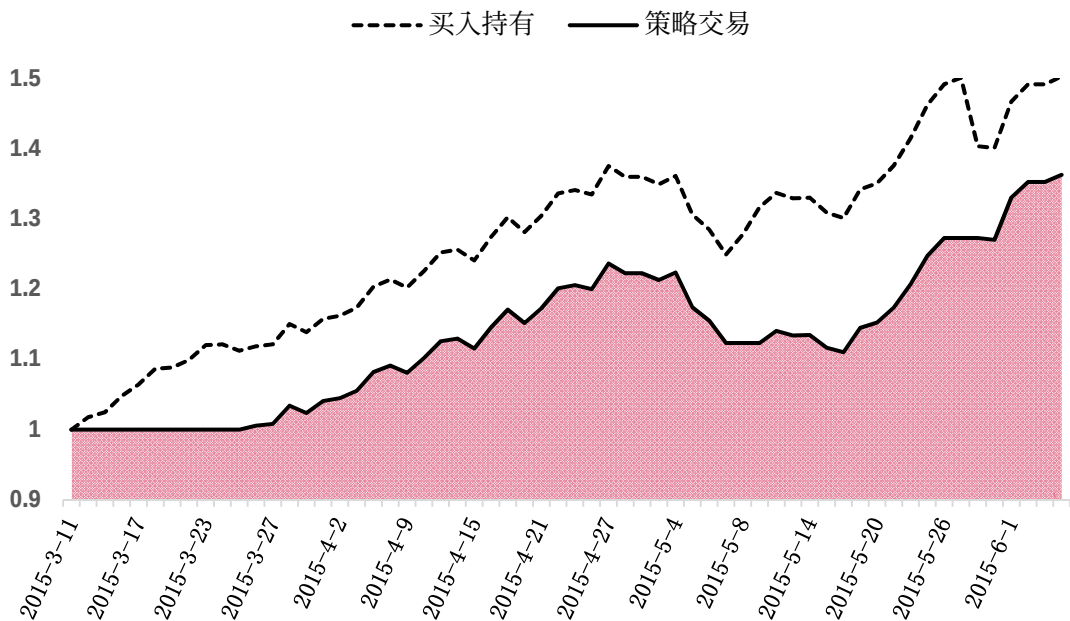


图 7：累计总资产变动情况图（2015-03-11 至 2015-06-04）

表 9：未加入指数上涨趋势区间回测结果

基本信息	预测天数：60	滑窗长度：0
交易类型	买入并持有	策略交易
初始资金	1 万元	1 万元
累计收益率	50.33%	36.30%
最终余额	1.5033 万元	1.3630 万元
年化收益率	447%	263%
最大回撤	-9.17%	-10.22%
最大连续亏损次数	3	6

在该具有明显上涨趋势的测试期，模型策略交易虽然没有跑赢上证指数，但也捕捉到了较大的收益空间，取得了 36.30%的累计收益率。

(3) 具有明显下跌趋势的回测期

在 2015 年 6 月 5 日至 2015 年 8 月 30 日的这段“股灾”期间，该模型的预测效果如图 8 和表 10 所示：

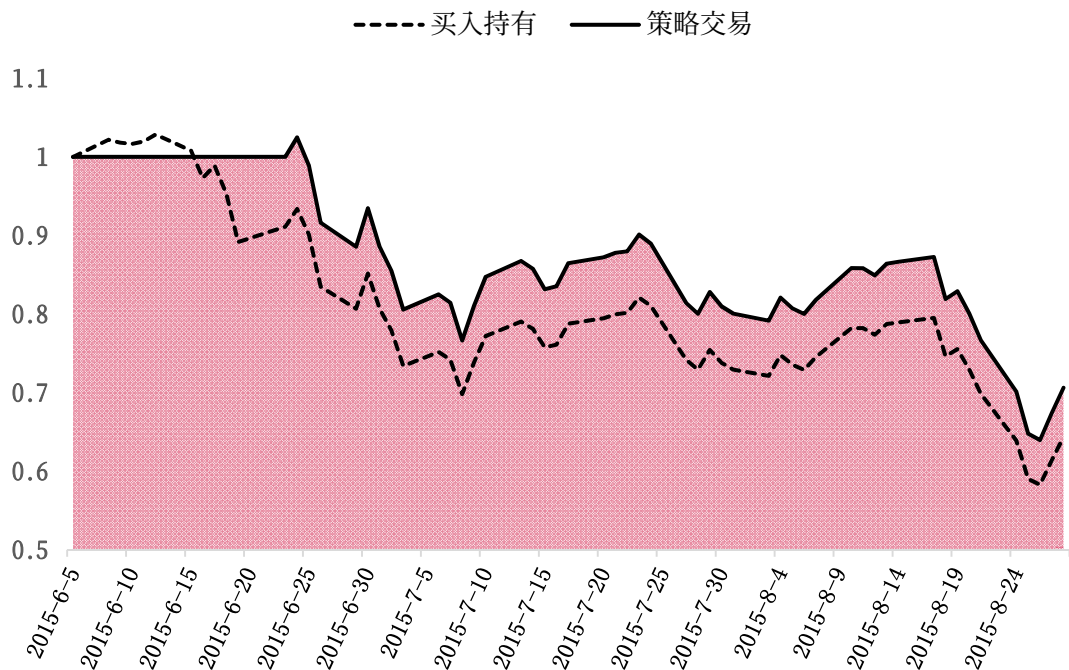


图 8：累计总资产变动情况图（2015-06-05 至 2015-08-31）

表 10：未加入指数下跌趋势区间回测结果

基本信息	预测天数：60	滑窗长度：0
交易类型	买入并持有	策略交易
初始资金	1 万元	1 万元
累计收益率	-35.65%	-29.37%
最终余额	0.6435 万元	0.7063 万元
年化收益率	-84.07%	-76.52%
最大回撤	-43.34%	-37.59%
最大连续亏损次数	5	5

该模型在这段测试期虽然没有正确地预测出市场未来的走势，但仍然以小幅优势跑赢了上证指数。

（4） 具有震荡趋势的回测期

下面我们对 2015 年 8 月 31 日至 2017 年 5 月 24 日这一具有震荡趋势的时间段进行单独考察，具体结果如图 9 和表 11 所示：

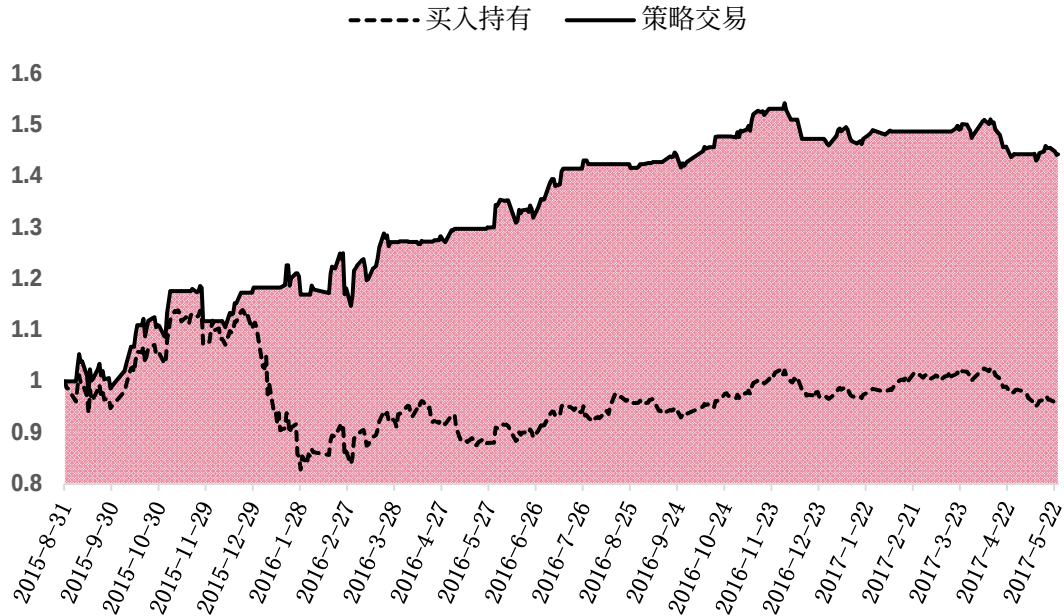


图 9：累计总资产变动情况图（2015-08-31 至 2017-05-24）

表 11：未加入指数震荡区间回测结果

基本信息	预测天数：420	滑窗长度：60
交易类型	买入并持有	策略交易
初始资金	1 万元	1 万元
累计收益率	-4.43%	44.27%
最终余额	0.9557 万元	1.4427 万元
年化收益率	-2.61%	24.38%
最大回撤	-27.28%	-8.23%
最大连续亏损次数	5	7

通过图 9 和表 11 反映的结果可以明显看出，本文所建立的择时模型在“震荡市”阶段对市场未来趋势的预测有着较高的精度，年化收益达到了 24.38%，最大回撤控制在了 8.23%，最大连续亏损次数也较少。而这段期间上证指数下跌了 4.43%，按照我们的模型所执行的交易决策明显好于买入并持有。

3. 加入投资者情绪指数的回测结果分析及评价

为探讨本文基于文本挖掘所建立的投资者情绪指数对模型预测性能是否有提升性能，我们将投资者情绪指数作为上证指数数据集的一个新的特征加入到 SVM 中。同样地，首先以 2015 年 3 月 11 日至 2017 年 5 月 24 日为回测期，选取初始资金为 1 万元，假设每次交易的费率为 0.03%，将加入投资者情绪指数后的模型结果与未加入情绪指数的情况作比较，然后对回测期进行趋势划分，分别对具有上涨、下跌和震荡趋势的测试时间段进行讨论。

(1) 整个回测期

2015 年 3 月 11 日至 2017 年 5 月 24 日期间，加入和未加入投资者情绪指数的股票市场择时交易模型的性能对比结果如图 10 和表 12 所示：

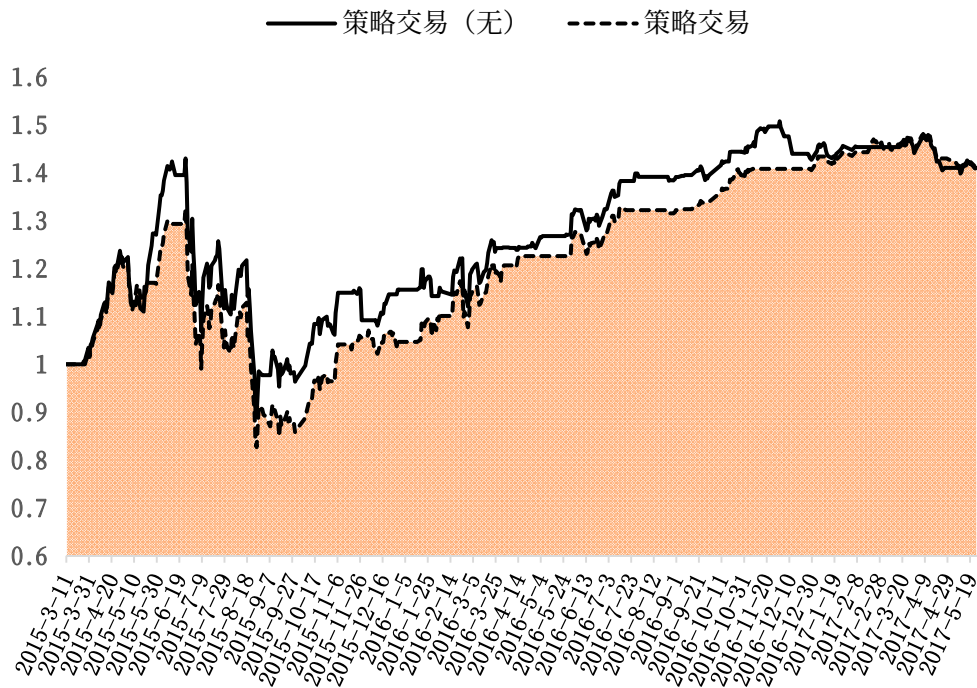


图 10：累计总资产变动情况图（2015-03-11 至 2017-05-24）

表 12：加入指数整个区间回测结果

基本信息	预测天数：540	滑窗长度：60
交易类型	加入投资者情绪指数	未加入投资者情绪指数
初始资金	1 万元	1 万元
累计收益率	40.69%	40.99%
最终余额	1.4069 万元	1.4099 万元
年化收益率	17.12%	17.24%
最大回撤	-37.59%	-37.59%
最大连续亏损次数	2	2

对于整个回测期，加入投资者情绪指数的股票市场择时交易模型的年化收益率、最大回撤等指标基本没有发生变化。

(2) 具有明显上涨趋势的回测期

在 2015 年 3 月 11 日至 2015 年 6 月 4 日这段具有明显上涨趋势的测试期，两者的对比结果如图 11 表 13 所示：

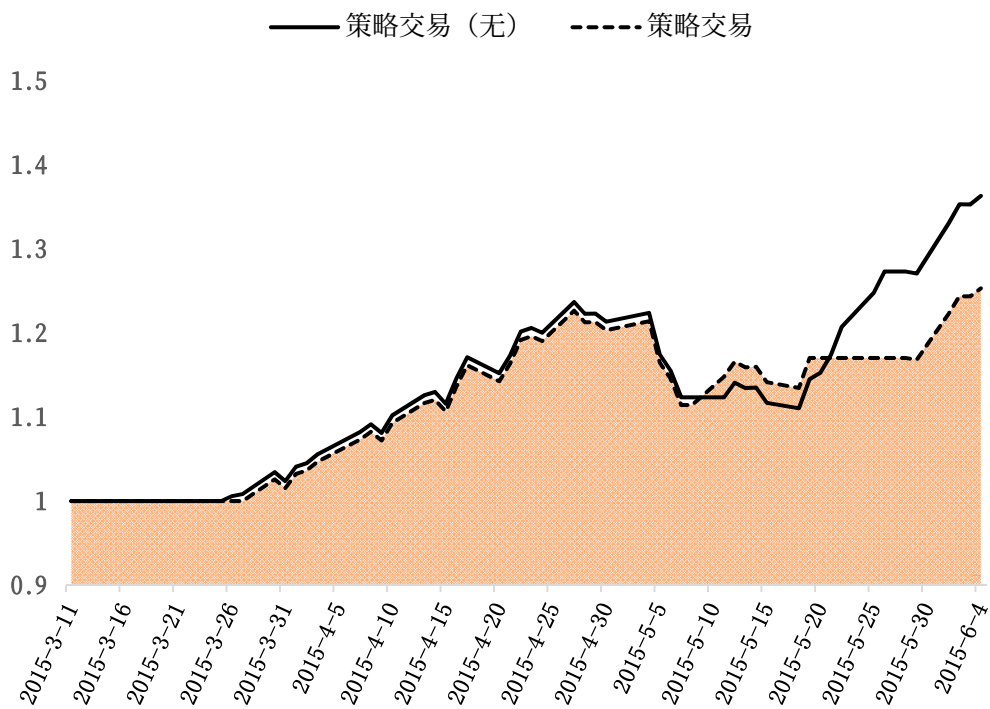


图 11：累计总资产变动情况图（2015-03-11 至 2017-06-04）

表 13：加入指数上涨趋势区间回测结果

基本信息	预测天数：60	滑窗长度：0
交易类型	加入投资者情绪指数	未加入投资者情绪指数
初始资金	1 万元	1 万元
累计收益率	25.29%	36.30%
最终余额	1.2529 万元	1.3630 万元
年化收益率	155.87%	263.44%
最大回撤	-9.17	-10.22%
最大连续亏损次数	5	6

虽然加入投资者情绪指数后，降低了该回测期的年化收益率，但该模型实现了最大回撤的降低，取得了较高的 25.29%的累计收益率。

(3) 具有明显下跌趋势的回测期

在 2015 年 6 月 5 日至 2015 年 8 月 30 日这段具有明显下跌趋势的回测期，加入与未加入情绪指数的模型对应的交易策略完全相同，对应的资产变动情况可参见图 8。

(4) 具有震荡趋势的回测期

同样地，最后单独考察 2015 年 8 月 31 日至 2017 年 5 月 24 日这一具有“震荡”趋势的时间段，具体结果如图 12 和表 14 所示：

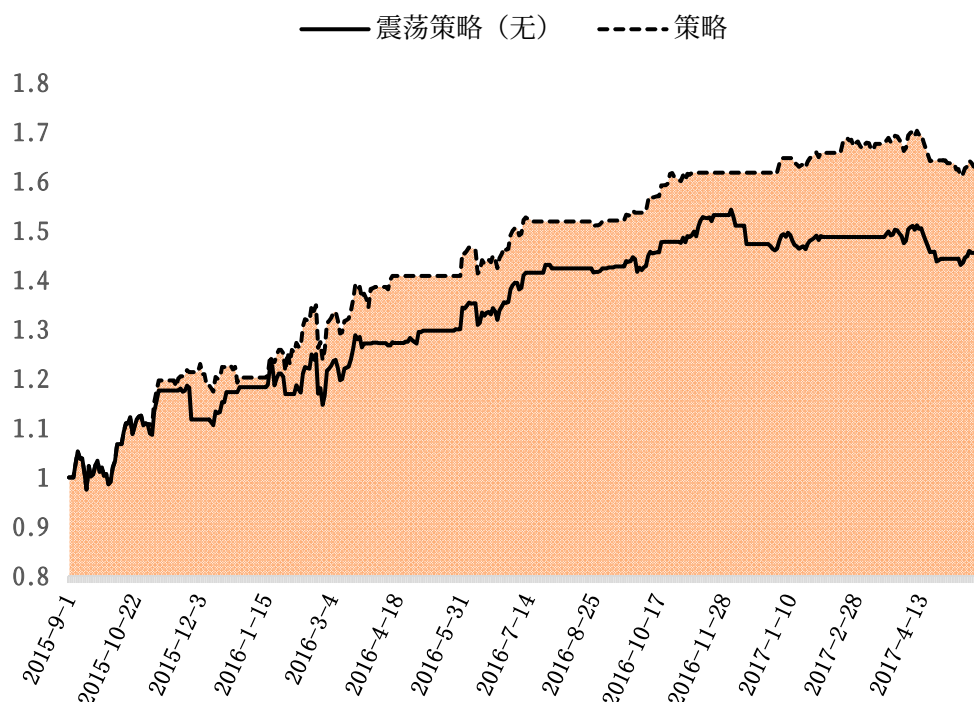


图 12：累计总资产变动情况图（2015-08-31 至 2017-05-24）

表 14：加入指数震荡区间回测结果

基本信息	预测天数：420	滑窗长度：60
交易类型	加入投资者情绪指数	未加入投资者情绪指数
初始资金	1 万元	1 万元
累计收益率	61.63%	44.27%
最终余额	1.6163 万元	1.4427 万元
年化收益率	33.08%	24.38%
最大回撤	-8.23%	-8.23%
最大连续亏损次数	4	7

从加入和未加入投资者情绪指数所分别对应的累积总资产的变动情况可以明显看出，在这一具有“震荡”趋势的测试阶段，加入投资者情绪指数后，本文所建立的择时交易模型明显取得了更好的效果，在保持最大回撤不变的前提下，年化收益率达到 33.08%。

六、结论和建议

（一）模型结论

在大数据和人工智能的背景下，机器学习的技术优势及其迅猛发展为股票市场相关投资交易策略的产生提供了一种新的思路。基于结构风险最小化原理和核

思想而建立的 SVM 方法, 在解决非线性和高维模式识别等问题中具有较优的预测性能, 对于股价波动剧烈、受非理性因素影响较大的中国股票市场, 其表现出许多特有的优势。

(1) 本文构建的基于 SVM 的股票市场择时交易模型在上涨市期间虽然没有跑赢上证指数, 但捕捉到了较高的 36.30% 的累积收益率; 在下跌市期间, 虽然没有正确地预测出市场未来的走势, 但仍然以小幅优势跑赢了上证指数; 在具有震荡趋势的市场阶段, 对于市场未来的趋势具有较高的预测精度, 取得了较高的 44.27% 的年化收益率, 且交易风险(最大回撤)较小。

(2) 当把基于文本挖掘所构建的投资者情绪指数作为 SVM 的一个训练特征时, 在上涨市期间预测性能略有下滑, 但仍取得了 17.12% 的累计收益率; 在下跌市期间, 其资产收益情况没有发生变化; 在震荡市期间, 相比未加入情绪指数的模型, 其准确地捕捉到了更多的价格波动信息, 在保证最大回撤不变的前提下, 取得了更高的累计收益率。综上, 本文构建的投资者情绪指数对于股票市场价格波动的信息反映是有效的。

(二) 模型改进建议

本文利用上证指数的历史数据, 以 SVM 为基础, 建立了股票市场择时交易模型, 同时探讨了基于文本挖掘构建的投资者情绪指数对于价格波动预测的有效性。这为大数据背景下, 机器学习技术用于量化投资提供了一种思路和方法。实际中可利用股票实盘数据接口对该策略进行模拟交易, 进一步验证该策略的有效性。

本文基于文本挖掘所构建的投资者情绪指数主要依据的是东方财富网“股吧”的文本和利用百度指数对关键词的扩充, 这些数据仅仅是体现股票价格波动的大数据的一部分, 其他结构化或非结构化的数据也能体现股价波动信息。因此, 可完善爬虫程序对数据的种类及量级进行扩充, 优化股市情绪关键词选取方案, 提高投资者情绪指数的有效性。

参考文献

- [1] Kalev P S., Liu W M., Pham P K., et al. Public information arrival and volatility of intraday stock returns[J]. Journal of Banking & Finance, 2004, 28(6):1441-1467
- [2] Antweiler W., Frank M Z. Is all that just noise? The information content of internet stock message boards[J]. The Journal of Finance, 2004, 59(3):1259-1294
- [3] Li F. The Information Content of Forward Looking Statements in Corporate

Filings-A Naïve Bayesian Machine Learning Approach[J]. Journal of Accounting Research, 2010, 48(5):1049-1102

[4] 张世军,程国胜,蔡吉花等. 基于网络舆情支持向量回归的股票价格预测研究[J]. 数学的实践与认识, 2013, 43(024):33-40

[5] Junqué de Fortuny E., De Smedt T. Martens D., et al. Evaluating and understanding text-based stock price prediction models[J]. Information Processing & Management, 2014

[6] 郭晓菲,李清军,潘子颖, 蒋峰. 基于投资者情绪指数的上证综指预测研究[C]. 2015年(第四届)全国大学生统计建模大赛, 2015: 1583-1606.

[7] Luís Torgo. 数据挖掘与R语言[M]. 北京: 机械工业出版社, 1915.67-115

[8] 欧阳红兵,彭浩彪. 量化投资技术与策略[M]. 北京: 北京大学出版社, 2016.

附录

此处仅列出部分代码，全部代码可参见附件。

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import traceback
import time
import urllib
import json
from datetime import datetime, timedelta

from selenium import webdriver
from threadpool import ThreadPool, makeRequests

from config import ini_config
from api import Api
from utils.log import logger

class BaiduBrowser(object):
    def __init__(self, cookie_json='', check_login=True):
        if not ini_config.browser_driver:
            browser_driver_name = 'Firefox'
        else:
            browser_driver_name = ini_config.browser_driver
        browser_driver_class = getattr(webdriver, browser_driver_name)

        if ini_config.browser_driver == 'Chrome':
            self.browser =
browser_driver_class(executable_path=ini_config.executable_path)
        else:
            self.browser = browser_driver_class()
        # 设置超时时间
        self.browser.set_page_load_timeout(50)
        # 设置脚本运行超时时间
        self.browser.set_script_timeout(10)
        # 百度用户名
        self.user_name = ini_config.user_name
        # 百度密码
        self.password = ini_config.password
        self.cookie_json = cookie_json
        self.api = None
        self.cookie_dict_list = []
```

```

self.date_list = []

self.init_api(check_login=check_login)
self.init_date_list()

def __del__(self):
    self.close()

def is_login(self):
    # 如果初始化 BaiduBrowser 时传递了 cookie 信息，则检测一下是否登录状态
    self.login_with_cookie(self.cookie_json)
    # 访问待检测的页面
    self.browser.get(ini_config.user_center_url)
    html = self.browser.page_source
    # 检测是否有登录成功标记
    return ini_config.login_sign in html

def init_api(self, check_login=True):
    # 判断是否需要登录
    need_login = False
    if not self.cookie_json:
        logger.info(u'因无历史 cookie，本次执行需要登录百度')
        need_login = True
    elif check_login and not self.is_login():
        logger.info(u'加载历史 cookie 登录失败，本次执行需要登录百度')
        need_login = True
    else:
        logger.info(u'本次执行无需登录百度')
    # 执行浏览器自动填表登录，登录后获取 cookie
    if need_login:
        self.login(self.user_name, self.password)
        self.cookie_json = self.get_cookie_json()
    cookie_str = self.get_cookie_str(self.cookie_json)
    # 获取到 cookie 后传给 api
    self.api = Api(cookie_str)

def get_date_info(self, start_date, end_date):
    # 如果 start_date 和 end_date 中带有“-”，则替换掉
    if start_date.find('-') != -1 and end_date.find('-') != -1:
        start_date = start_date.replace('-', '')
        end_date = end_date.replace('-', '')
    # start_date 和 end_date 转换成 datetime 对象
    start_date = datetime.strptime(start_date, '%Y%m%d')
    end_date = datetime.strptime(end_date, '%Y%m%d')

```

```

# 循环 start_date 和 end_date 的差值, 获取区间内所有的日期
date_list = []
temp_date = start_date
while temp_date <= end_date:
    date_list.append(temp_date.strftime("%Y-%m-%d"))
    temp_date += timedelta(days=1)
start_date = start_date.strftime("%Y-%m-%d")
end_date = end_date.strftime("%Y-%m-%d")
return start_date, end_date, date_list

def get_one_day_index(self, date, url, keyword, type_name, area):
    try_num = 0
    try_max_num = 5
    while try_num < try_max_num:
        try:
            try_num += 1
            # 获取图片的下载地址以及图片的切割信息
            img_url, val_info = self.api.get_index_show_html(url)
            # 下载 img 图片, 然后根据 css 切割图片的信息去切割图片, 组成新的图
            # 片,
            # 将新图片跟已经做好的图片识别库对应识别
            value = self.api.get_value_from_url(img_url, val_info)
            break
        except:
            pass
    logger.info(
        'keyword:%s, type_name:%s, area:%s, date:%s, value:%s' % (
            keyword, type_name, area, date, value
        )
    )
    return value.replace(',', '')

def get_baidu_index_by_date_range(self, keyword, start_date, end_date,
                                   type_name, area):
    # 根据区间获取关键词的索引值
    url = ini_config.time_range_trend_url.format(
        start_date=start_date, end_date=end_date,
        word=urllib.quote(keyword.encode('gbk')),
        area=area
    )
    self.browser.get(url)
    if ini_config.browser_sleep:
        time.sleep(float(ini_config.browser_sleep))

```

```

if u'未被收录' in self.browser.page_source:
    return {}
# 执行 js 获取后面所需的 res 和 res2 的值
res = self.browser.execute_script('return PPval.ppt;')
res2 = self.browser.execute_script('return PPval.res2;')

# 获取指定区间的日期列表,方便下面循环用
start_date, end_date, date_list = self.get_date_info(
    start_date, end_date
)

# 拼接 api 的 url
url = ini_config.all_index_url.format(
    res=res, res2=res2, start_date=start_date, end_date=end_date
)
# 获取 api 的结果信息,这里面保存了后面日期节点的一些加密值
all_index_info = self.api.get_all_index_html(url)
indexes_enc = all_index_info['data'][type_name][0]['userIndexes_enc']
enc_list = indexes_enc.split(',')

pool = ThreadPool(int(ini_config.num_of_threads))
# wm = WorkManager(int(ini_config.num_of_threads))

# 遍历这些 enc 值,这些值拼接出 api 的 url(这个页面返回 图片信息以及 css 规定的切图信息)
list_of_args = []
for index, _ in enumerate(enc_list):
    url = ini_config.index_show_url.format(
        res=res, res2=res2, enc_index=_, t=int(time.time()) * 1000
    )
    # 根据 enc 在列表中的位置,获取它的日期
    date = date_list[index]
    # 将任务添加到多线程下载模型中
    item = (None, dict(date=date, url=url, keyword=keyword,
type_name=type_name, area=area))
    list_of_args.append(item)

baidu_index_dict = {}

def callback(*args, **kwargs):
    req, val = args[0], args[1]
    baidu_index_dict[req.kwds['date']] = val

req_list = makeRequests(self.get_one_day_index, list_of_args, callback)

```

```
[pool.putRequest(req) for req in req_list]
pool.wait()

return baidu_index_dict

def _get_index_period(self, keyword, area):
    # 拼接一周趋势的 url
    url = ini_config.one_week_trend_url.format(
        area=area, word=urllib.quote(keyword.encode('gbk'))
    )
    self.browser.get(url)
    # 获取下方 api 要用到的 res 和 res2 的值
    res = self.browser.execute_script('return PPval.ppt;')
    res2 = self.browser.execute_script('return PPval.res2;')
    start_date, end_date = self.browser.execute_script(
        'return BID.getParams.time()[0];'
    ).split('|')
    start_date, end_date, date_list = self.get_date_info(
        start_date, end_date
    )
    url = ini_config.all_index_url.format(
        res=res, res2=res2, start_date=start_date, end_date=end_date
    )
    all_index_info = self.api.get_all_index_html(url)
    start_date, end_date = all_index_info['data']['all'][0][
        'period'].split('|')
    # 重置 start_date, end_date, 以 api 返回的为准
    start_date, end_date, date_list = self.get_date_info(
        start_date, end_date
    )
    logger.info(
        'all_start_date:%s, all_end_date:%s' % (start_date, end_date)
    )
    return date_list

def init_date_list(self):
    if ini_config.start_date and ini_config.end_date:
        _, _, date_list = self.get_date_info(
            start_date=ini_config.start_date, end_date=ini_config.end_date
        )
    else:
        # 配置文件不配置 start_date 和 end_date, 就会按照索引的最大区间来,
        # 目前每个关键词的最大区间是一样的, 都是从 2011 年 1 月 1 日开始的
        date_list = self._get_index_period(keyword=u'test', area=0)
```

```

self.date_list = date_list

def get_baidu_index(self, keyword, type_name, area):
    baidu_index_dict = dict()
    start = 0
    skip = 180
    end = len(self.date_list)
    while start < end:
        try:
            start_date = self.date_list[start]
            if start + skip >= end - 1:
                end_date = self.date_list[-1]
            else:
                end_date = self.date_list[start + skip]
            result = self.get_baidu_index_by_date_range(
                keyword, start_date, end_date, type_name, area
            )
            baidu_index_dict.update(result)
            start += skip + 1
        except:
            import traceback
            print traceback.format_exc()
    return baidu_index_dict

def login(self, user_name, password):
    login_url = ini_config.login_url
    # 访问登陆页
    self.browser.get(login_url)
    time.sleep(2)

    # 自动填写表单并提交, 如果出现验证码需要手动填写
    while 1:
        try:
            user_name_obj = self.browser.find_element_by_id(
                'TANGRAM_PSP_4__userName'
            )
            break
        except:
            logger.error(traceback.format_exc())
            time.sleep(1)
    user_name_obj.send_keys(user_name)
    ps_obj = self.browser.find_element_by_id('TANGRAM_PSP_4__password')
    ps_obj.send_keys(password)
    sub_obj = self.browser.find_element_by_id('TANGRAM_PSP_4__submit')

```

```

sub_obj.click()

# 如果页面的 url 没有改变, 则继续等待
while self.browser.current_url == login_url:
    time.sleep(1)

def close(self):
    if getattr(self, 'browser'):
        if self.browser:
            try:
                self.browser.quit()
            except:
                pass

def get_cookie_json(self):
    return json.dumps(self.browser.get_cookies())

def get_cookie_str(self, cookie_json=''):
    if cookie_json:
        cookies = json.loads(cookie_json)
    else:
        cookies = self.browser.get_cookies()
    return '; '.join(['%s=%s' % (item['name'], item['value'])
                      for item in cookies])

def login_with_cookie(self, cookie_json):
    self.browser.get('https://www.baidu.com/')
    for item in json.loads(cookie_json):
        try:
            self.browser.add_cookie(item)
        except:
            continue

import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn import preprocessing

#####1440
data_try = sw[11:-10] # 2188 行
data_try = data_try[data_try.date>='2011-03-24'] # 1500
data_try.to_csv('E:\Data Mining And Machine
Learning\dataset\stocks\index_new_day10_data_try2.csv', index=None)
data_try = pd.read_csv('E:\Data Mining And Machine

```

```

Learning\dataset\stocks\index_new_day10_data_try2.csv')
del data_try['future_cond_sum']
del data_try['date']
for i in np.arange(1,11, 1):
    column_name = 'future_rate_%s' % i
    del data_try[column_name]
## (1500-240-60)/60.0 = 20.0
time = 12
vali_index = []
for i in np.arange(0,time,1):
    vali_index.append(60*i)
print vali_index
tuned_parameters = {'gamma': [2**-15, 2**-13, 2**-11, 2**-9, 2**-7, 2**-5, 2**-3,
2, 2**3],
                    'C': [2**-5, 2**-3, 2**-1, 2, 2**3, 2**5, 2**7, 2**9, 2**11,
2**13, 2**15]}
pt = np.zeros( (tuned_parameters['gamma'].__len__(),tuned_parameters['C'].__len__()) )
# 9*11
retrace = np.zeros( (tuned_parameters['gamma'].__len__(),tuned_parameters['C'].__len__()) )
# 9*11
for j in vali_index:
    cv_train = data_try.loc[j:j+239,:]
    cv_vali = data_try.loc[j+240:j+299,:]
    cv_train_y = cv_train['label']
    cv_train_y = np.array(cv_train_y)
    cv_train_y = np.array([int(s) for s in cv_train_y])
    cv_train_x = cv_train.copy()
    del cv_train_x['label']
    cv_train_x = np.array(cv_train_x)
    cv_train_x_scaled = preprocessing.scale(cv_train_x)

    cv_vali_x = cv_vali.copy()
    del cv_vali_x['label']
    cv_vali_x = np.array(cv_vali_x)
    cv_vali_x_scaled = preprocessing.scale(cv_vali_x)
    for g in range(tuned_parameters['gamma'].__len__()):
        for c in range(tuned_parameters['C'].__len__()):
            clf = SVC(decision_function_shape='ovo', C=tuned_parameters['C'][c],
gamma=tuned_parameters['gamma'][g], kernel='rbf')
            clf.fit(cv_train_x_scaled, cv_train_y)
            l_tem = clf.predict(cv_vali_x_scaled)
            l = pd.Series([i for i in l_tem])

```

```

        price = pd.Series([s for s in cv_vali['close']])
        pt[g,c] += cum_profit(1,price)
        money = total_money(1, price)
        money_nona = pd.Series([i for i in money.dropna()])
        if max_retracement(money_nona)<=0:
            retrace[g,c] += max_retracement(money_nona)
        else:
            retrace[g,c] += 0
    number = vali_index.index(vali_index[-1])
    pt_ave = pt/(number+1.0)
    retrace_ave = retrace/(number+1.0)

pt_ave_list = [pt_ave[i][j] for i in range(9) for j in range(11)]
pt_ave_list_sorted = sorted(pt_ave_list,reverse=True)[:5]
print pt_ave_list_sorted

index = []
for i in pt_ave_list_sorted:
    index.append( pt_ave_list.index(i) )
print index

retrace_ave_list = []
index_trans = []
for k in index:
    g = location(k)[0]
    c = location(k)[1]
    index_trans.append((g,c))
    retrace_ave_list.append( retrace_ave[g,c] )
print index_trans
print retrace_ave_list

# 回测
train = data_try.loc[time*60:time*60+239,:]
test = data_try.loc[time*60+240:time*60+299,:]

train_y = train['label']
train_y = np.array(train_y)
train_y = np.array([int(s) for s in train_y])
train_x = train.copy()
del train_x['label']
train_x = np.array(train_x)
train_x_scaled = preprocessing.scale(train_x)

test_x = test.copy()

```

```

del test_x['label']
test_x = np.array(test_x)
test_x_scaled = preprocessing.scale(test_x)

clf = SVC(decision_function_shape='ovo', gamma=tuned_parameters['gamma'][2],
C=tuned_parameters['C'][9], kernel='rbf')
clf.fit(train_x_scaled, train_y)
l_tem = clf.predict(test_x_scaled)
l = pd.Series([i for i in l_tem])
price = pd.Series([s for s in test['close']])
print cum_profit(l,price)
money = total_money(l,price)
money_nona = pd.Series( [i for i in money.dropna()] )
print 'max_retracement:',max_retracement(money_nona)

from sklearn.linear_model import ElasticNet,LinearRegression
import numpy as np
import pandas as pd
bd_index = pd.read_csv(u'E:/Data Mining And Machine Learning/量化投资/统计建模比赛/文本挖掘/index_add_final.csv')

```